



UNIDADE I

# Garantia de Qualidade de Software (Parte I)



**Profa. Ana Carolina Gondim Inocêncio**

## Técnicas de Revisão — nem toda revisão é igual

- ✓ Revisões informais (teste de mesa, pair programming) já valem a pena, mesmo sem planejamento extenso.
- ✓ A Reunião de Término Formal (RTF) tem papéis, restrições e três decisões possíveis claras.
- ✓ Diretriz mais importante: revise o produto, não o produtor.
- ✓ Fechamos com a distinção RTF × Auditoria — revisão técnica é de engenheiros para engenheiros; auditoria é feita pelo pessoal de SQA.

*Hoje: quem faz a auditoria, e como o SQA garante que as diretrizes de qualidade sejam seguidas.*

# Garantia de Qualidade de Software — Parte I

- 1 Introdução — o que é Garantia de Qualidade de Software (SQA)
- 2 Elementos de Garantia da Qualidade de Software
- 3 Tarefas do Grupo de SQA
- 4 SQA na prática moderna — CI/CD, DevSecOps e quality gates
- 5 Kahoot valendo nota (AC1)

POLL EVERYWHERE · RESPONDAM PELO CELULAR

**Quem vocês acham que é responsável por garantir que as diretrizes de qualidade sejam seguidas do início ao fim do projeto — não só nas reuniões de revisão?**

**PoolEveryWhere**

Vamos projetar as respostas na tela em tempo real — sem certo ou errado ainda.

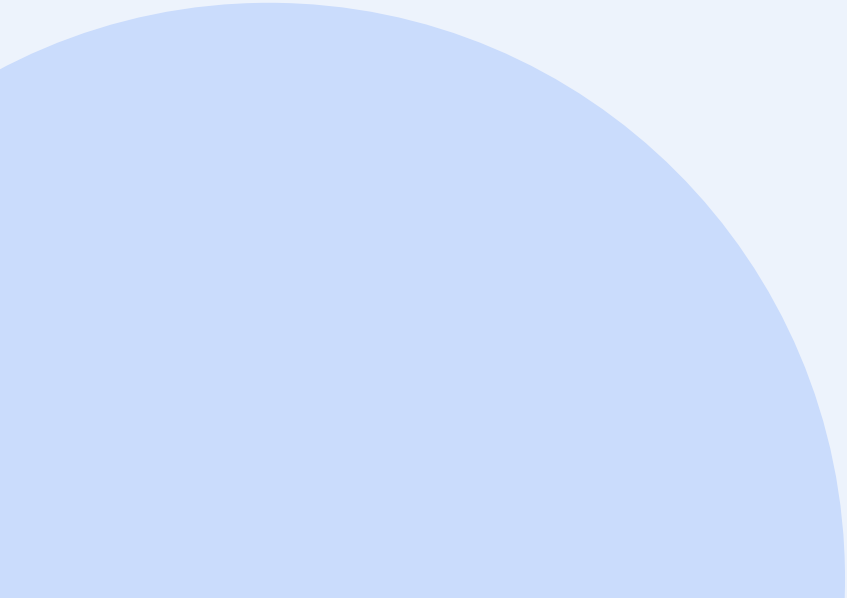
## Ligando com a Aula 4

- A RTF é uma técnica pontual: revisa um artefato específico, num momento específico.
- A auditoria — feita pelo pessoal de SQA — é uma entre muitas frentes de um trabalho contínuo, do primeiro requisito ao último release.
- Hoje vamos conhecer esse trabalho contínuo: a Garantia da Qualidade de Software (SQA).



**01**

# **Introdução**



## O Problema Real, Segundo Crosby

“O problema da gestão da qualidade não é o que as pessoas não sabem a seu respeito. Mas sim, o que elas pensam que sabem.”

*Philip Crosby, um dos fundadores do movimento moderno de qualidade.*

# Qualidade Não Começa Depois do Código

Alguns desenvolvedores ainda acreditam que qualidade de software é algo sobre o qual só começamos a nos preocupar depois que o código é gerado. Nada poderia estar mais distante da verdade.

***SQA (Software Quality Assurance) é uma atividade universal, aplicada a toda a gestão da qualidade.***

# Sete Perguntas Que Vamos Responder

- O que é?
- Quem realiza?
- Por que é importante?
- Quais são as etapas envolvidas?
- Qual é o artefato?
- Como garantir que o trabalho foi feito corretamente?
- Qual é o objetivo?

# Três Respostas Diretas

## O que é?

Definir explicitamente o que se quer dizer com “qualidade de software”, criar atividades que garantam alta qualidade em todo artefato, e usar métricas para melhorar a gestão da qualidade.

## Quem realiza?

Todos os envolvidos no processo de engenharia de software são responsáveis pela qualidade — não só um departamento.

## Por que é importante?

Pode-se fazer certo da primeira vez, ou então fazer tudo novamente.

# Mais Três Respostas

## Quais as etapas?

Definir qualidade em diferentes níveis de abstração; depois, definir as atividades de SQA que vão filtrar erros antes que sigam adiante.

## Qual é o artefato?

O plano de garantia da qualidade de software, que define a estratégia de SQA da equipe.

## Como garantir corretude?

Trabalhar pela eficiência na remoção de defeitos, reduzindo a quantidade de reformulações necessárias.

## Qual É o Objetivo, Afinal?

Concentrar-se em problemas de gerenciamento e em atividades específicas de processo que permitam a uma organização de software fazer “as coisas certas, no momento certo e da maneira certa”.

*Esse é o fio condutor de tudo que vamos ver hoje.*

POLL EVERYWHERE · RESPONDAM PELO CELULAR

**Na prática de vocês, quem deveria ser o “dono” da qualidade em um projeto de software?**

*Retomando a pergunta “Quem realiza?”*

**PollEveryWhere**

Vamos projetar as respostas na tela em tempo real — sem certo ou errado ainda.

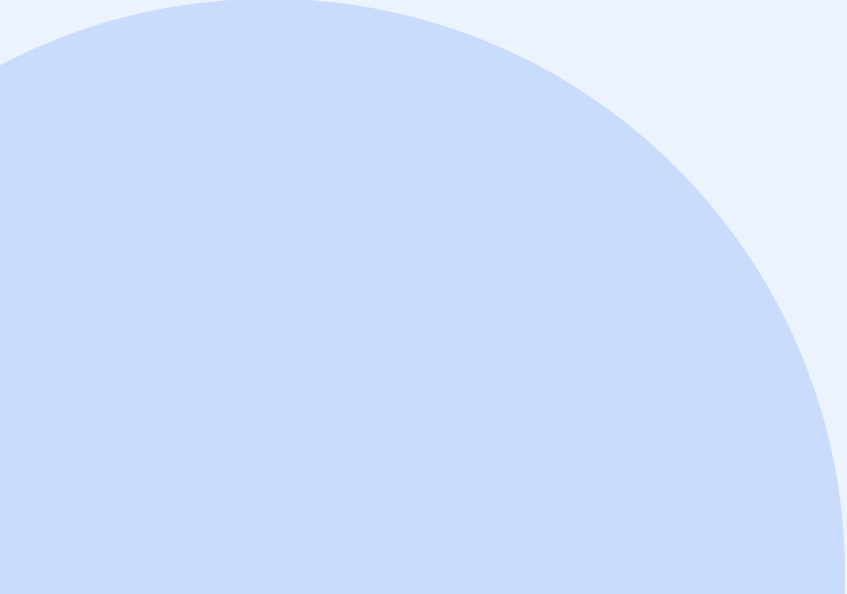
## Todo Mundo, o Tempo Todo

- Não existe um único “dono” da qualidade — ela é responsabilidade compartilhada por toda a equipe de engenharia de software.
- Isso não elimina a necessidade de um grupo especializado: é aí que entra o grupo de SQA, que veremos mais adiante.
- Agora vamos aos elementos concretos que a garantia da qualidade de software abrange.



**02**

## **Elementos de Garantia da Qualidade de Software**



## Um Amplo Espectro de Preocupações

A garantia da qualidade de software engloba um amplo espectro de preocupações e atividades que se concentram na gestão da qualidade de software.

# Nove Frentes de Atuação

- Padrões
- Revisões e Auditorias
- Testes
- Coleta e Análise de Erros e Defeitos
- Gerenciamento de Mudanças
- Educação
- Gerência de Fornecedores
- Segurança e Proteção
- Administração de Riscos

# 1–3: Padrões, Revisões e Testes

## **Padrões**

IEEE, ISO e outras organizações produzem padrões para engenharia de software. Podem ser adotados voluntariamente ou exigidos pelo cliente.

## **Revisões e Auditorias**

Revisões técnicas são de engenheiros para engenheiros, para revelar erros. Auditorias são feitas pelo SQA, para garantir que as diretrizes de qualidade estão sendo seguidas.

## **Testes**

Função de controle de qualidade cujo objetivo principal é descobrir erros. O SQA garante que os testes sejam planejados e conduzidos com eficiência.

# 4–6: Erros, Mudanças e Educação

## Coleta e Análise de Erros

O SQA reúne e analisa dados de erros e defeitos para entender como são introduzidos e quais atividades de ES melhor os eliminam.

## Gerenciamento de Mudanças

Mudanças mal administradas geram confusão — e confusão quase sempre leva a uma qualidade inadequada.

## Educação

Melhorar a prática de engenharia de software depende de educar engenheiros, gerentes e interessados — não é tarefa de um único idealista.

# 7–9: Fornecedores, Segurança e Riscos

## Gerência de Fornecedores

O SQA sugere práticas de qualidade ao fornecedor e incorpora exigências de qualidade em contratos com fornecedores externos.

## Segurança e Proteção

O SQA garante processos e tecnologias adequados à segurança desejada, avalia o impacto de falhas e inicia etapas de redução de risco.

## Administração de Riscos

O SQA garante que a gestão de riscos seja conduzida apropriadamente e que planos de contingência estejam estabelecidos.

## Além Disso

A SQA também trabalha para garantir que as atividades de suporte ao software — manutenção, suporte on-line, documentação e manuais — sejam realizadas tendo a qualidade como preocupação dominante.

## As Invisíveis Costumam Falhar Primeiro

- Padrões e testes recebem atenção porque são tangíveis — dá para apontar um padrão descumprido ou um teste que falhou.
- Educação e gerenciamento de mudanças são discretos, quase invisíveis — até que sua ausência causa um problema grande.
- É exatamente aí que a qualidade costuma desandar: no que ninguém está observando de perto.



**03**

## **Tarefas do Grupo de SQA**

## Dois Elementos Distintos

A garantia da qualidade de software é composta por tarefas associadas a dois elementos distintos: os engenheiros de software que realizam o trabalho técnico, e o grupo de SQA, responsável pelo planejamento, supervisão, manutenção de registros, análise e relatórios.

# O Que o Grupo de SQA Faz

- Prepara um plano de SQA para o projeto
- Participa do desenvolvimento da descrição de processo do projeto
- Revisa as atividades de engenharia de software para verificar conformidade
- Audita produtos de software resultantes designados
- Garante que desvios sejam documentados e tratados
- Registra qualquer não aderência e relata à gerência superior

# As Duas Primeiras Tarefas, em Detalhe

## **Plano de SQA**

Desenvolvido como parte do planejamento do projeto e revisado por todos os interessados.

## **Descrição do processo**

A equipe seleciona o processo de trabalho; o SQA revisa a conformidade com a política organizacional e padrões (ex.: ISO-9001).

# Revisão, Auditoria e Tratamento de Desvios

## **Revisa atividades**

Identifica, documenta e acompanha desvios do processo definido, verificando se as correções foram feitas.

## **Audita produtos**

Revisa produtos selecionados; identifica e documenta desvios; relata periodicamente ao gerente de projeto.

## **Garante tratamento**

Desvios no plano, na descrição do processo, nos padrões ou no artefato seguem um procedimento documentado.

## A Última Linha de Defesa

O grupo de SQA registra qualquer não aderência e relata ao gerenciamento superior: itens com problemas são acompanhados até que sejam resolvidos.

*O SQA não aponta o erro e vai embora — ele acompanha até o fechamento.*

**Vocês já viram, num projeto acadêmico ou estágio, alguém com esse papel de acompanhar desvios até o fim? O que acontece quando ninguém tem esse papel?**

*Em duplas — 2 minutos para conversar. Depois, 2-3 duplas compartilham com a turma.*



**04**

## **SQA na Prática Moderna**



## O Mesmo Princípio, Décadas Depois

Os elementos e tarefas que vimos vêm de Pressman e das recomendações do SEI, mas o princípio central — qualidade não é uma fase, é uma atividade contínua — está mais vivo do que nunca nas equipes de DevSecOps(**D**evelopment (desenvolvimento) + **S**ecurity (segurança) + **O**perations (operações)).

*SQA não desapareceu. Boa parte dele virou código.*

# SQA Tradicional × SQA em CI/CD

## O modelo tradicional

- Auditorias e revisões concentradas em marcos do projeto
- Grupo de SQA separado da equipe de desenvolvimento
- Feedback de qualidade em dias ou semanas

## O modelo em CI/CD

- *Quality gates* automatizados a cada *commit* ou *pipeline*
- Qualidade é responsabilidade compartilhada (“*shift-left*”)
- Feedback de qualidade em minutos

# O Que É um Pipeline?

Pense numa linha de montagem de fábrica: o código entra de um lado, passa por uma sequência de estações automáticas, e só sai pronto do outro lado se passar em todas elas.



*Cada estação (“estágio”) roda sozinha, sem uma pessoa precisar clicar em nada. Se o código falhar em qualquer uma delas, o pipeline para ali — ele não segue pro próximo estágio.*

# CI/CD, Estágio por Estágio

CI e CD são dois pedaços diferentes do mesmo pipeline que vimos no slide anterior — CI cuida do começo, CD cuida do fim.

## CI — Integração Contínua

- Cada **commit** dispara o **pipeline** sozinho
- O código é compilado (**“build”**)
- Os testes automatizados rodam
- Objetivo: pegar erros de integração cedo

## CD — Entrega/Implantação Contínua

- **Continuous Delivery**: fica pronto para lançar — falta um clique humano
- **Continuous Deployment**: vai para produção sozinho, sem clique
- Objetivo: reduzir o tempo entre “pronto” e “em uso”

# Quality Gates: os Portões da Qualidade

Um quality gate é um ponto de checagem automático que o código PRECISA passar para seguir adiante no pipeline. Se falhar, o pipeline para — ninguém decide isso caso a caso, é a regra que decide.



## Análise estática

zero *code smells* (sinal de que algo no código não está bem estruturado) críticos, cobertura mínima de testes



## Testes automatizados

unitários e de integração passam, sem regressões



## Varredura de segurança

zero vulnerabilidades críticas (CVEs) nas dependências



## Aprovação de *code review*

pelo menos um colega revisou o código antes do merge

# O Custo de Adiar a Qualidade

**US\$ 1**

para corrigir um defeito ainda na fase de design

**US\$ 100**

para corrigir o mesmo defeito depois de estar em produção

*Mesma lição da amplificação de defeitos que vimos na Aula 3, só que agora automatizada em cada pipeline de CI/CD.*

# Shift-Left: Por Que “Deslocar Para a Esquerda”?

Imagine a linha do tempo de um projeto, da esquerda (início) para a direita (produção):



**Shift-left não é uma ferramenta específica — é um princípio:** quanto mais cedo a checagem de qualidade acontece, mais barato é consertar o que ela encontra.

*Lembra do slide anterior “O Custo de Adiar a Qualidade” (US\$1 no design × US\$100 em produção)? É exatamente essa lógica que justifica mover a checagem para a esquerda.*

RESPONDAM COMPARTILHANDO COM OS COLEGAS

**Vocês já viram, ou usaram, alguma ferramenta (SonarQube, GitHub Actions, GitLab CI) bloquear um merge por causa de qualidade?**

**Levantem a mão e expliquem quando e qual ferramenta.**

## O Auditor Virou Parte do Pipeline

- O grupo de SQA de Pressman não sumiu — boa parte do seu trabalho de verificação foi automatizada.
- Um quality gate faz, em segundos, parte do que um auditor humano levaria dias para checar manualmente.
- Mas atenção: automação verifica conformidade com regras — o **JULGAMENTO** sobre “o que é qualidade suficiente” continua sendo humano, ou seja, são as pessoas que escrevem as regras.

# Conclusão

- SQA é uma atividade universal — não começa depois que o código é gerado
- Qualidade é responsabilidade de todos os envolvidos, apoiada por um grupo de SQA dedicado
- As tarefas do grupo de SQA vão do planejamento à auditoria e ao acompanhamento de não aderências
- Na prática moderna, boa parte desse trabalho está automatizada em quality gates de CI/CD

# Duas Perguntas Para Fechar a Aula

- É possível avaliar a qualidade de software se o cliente ficar alterando continuamente aquilo que supostamente deveria estar pronto?
- Um programa pode ser correto e, ainda assim, não ser confiável? Expliquem.

## Próxima Aula

Garantia de Qualidade de Software (Parte II): daremos continuidade aos temas de SQA, aprofundando métricas e o plano de garantia da qualidade.

# OBRIGADA PELA ATENÇÃO

Questionário Gamificado — Garantia de Qualidade de Software (Parte I)

## QUESTIONÁRIO GAMIFICADO

**Kahoot**